```
#include <iostream>
#include <signal.h>
#include <unistd.h>
#include <fcntl.h>
#include <sstream>
#include <chrono>
#include <pigpio.h>
#include <string>
#include <cerrno>
#include <cstring>

#include <sys/ioctl.h>
#include <asm/termbits.h>

#include "sl_lidar.h"
#include "sl_lidar_driver.h"

using namespace sl;

// ====================================================
// GLOBALS
// ====================================================
ILidarDriver* lidar = nullptr;
sl::IChannel* lidarChannel = nullptr;
int motor_fd = -1;
int xbee_fd  = -1;

const int LOOP_PERIOD_MS = 50;

// Flag d'arrêt propre
volatile sig_atomic_t stopRequested = 0;

// ====================================================
// millis()
// ====================================================
unsigned long millis() {
    using namespace std::chrono;
    static steady_clock::time_point start = steady_clock::now();
    return (unsigned long)std::chrono::duration_cast<std::chrono::milliseconds>(
               std::chrono::steady_clock::now() - start)
        .count();
}

// ====================================================
// CONFIG SERIAL
// ====================================================
void configSerialPort(int fd, int baudrate) {
    struct termios2 tio;
```

```
    ioctl(fd, TCGETS2, &tio);

    tio.c_cflag &= ~CBAUD;
    tio.c_cflag |= BOTHER;
    tio.c_ispeed = baudrate;
    tio.c_ospeed = baudrate;

    tio.c_cflag &= ~CRTSCTS;
    tio.c_cflag |= CREAD | CLOCAL;
    tio.c_cflag &= ~PARENB;
    tio.c_cflag &= ~CSTOPB;
    tio.c_cflag &= ~CSIZE;
    tio.c_cflag |= CS8;

    tio.c_cc[VTIME] = 0;
    tio.c_cc[VMIN]  = 0;

    ioctl(fd, TCSETS2, &tio);
}

// ========================================================
// XBEE PARSER
// ========================================================
bool readXbeeLine(int fd, std::string& lineOut) {
    static std::string buffer;
    char c;

    while (read(fd, &c, 1) == 1) {

        if ((unsigned char)c < 0x09 || ((unsigned char)c > 0x0D && (unsigned char)c < 0x20)) {
            std::cout << "[XBEE RAW] octet non imprimable reçu → XBee peut être en mode
API" << std::endl;
        }

        if (c == '\r') continue;

        if (c == '\n') {
            if (!buffer.empty()) {
                lineOut = buffer;
                buffer.clear();
                return true;
            }
            continue;
        }

        buffer += c;
```

```
        if (buffer.size() > 256) {
            std::cout << "[XBEE] buffer overflow → reset" << std::endl;
            buffer.clear();
        }
    }

    return false;
}

void resetXbeeParser() {
    std::string dummy;
}

// ======================================================
// CTRL+C
// ======================================================
void ctrlc_handler(int) {
    stopRequested = 1;
}

// ======================================================
// ULTRASON
// ======================================================
float readGroveUltrasonic(int pin)
{
    gpioSetMode(pin, PI_OUTPUT);
    gpioWrite(pin, 0);
    gpioDelay(2);

    gpioWrite(pin, 1);
    gpioDelay(10);
    gpioWrite(pin, 0);

    gpioSetMode(pin, PI_INPUT);

    uint32_t start_wait = gpioTick();
    while (gpioRead(pin) == 0) {
        if (gpioTick() - start_wait > 30000) return -1;
    }

    uint32_t high_start = gpioTick();
    while (gpioRead(pin) == 1) {
        if (gpioTick() - high_start > 30000) return -1;
    }
    uint32_t high_end = gpioTick();

    uint32_t duration = high_end - high_start;
    return duration / 58.0f;
```

```
}

// ====================================================
// NON-BLOCKING MOTOR WRITE
// ====================================================
void sendMotorCommand(int fd, const std::string& msg) {
    if (fd < 0) return;
    ssize_t n = write(fd, msg.c_str(), msg.size());
    if (n < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            std::cout << "[MOTOR] write EAGAIN, skipping frame" << std::endl;
        } else {
            std::cout << "[MOTOR] write error: " << strerror(errno) << std::endl;
        }
    }
}

// ====================================================
// SESSION CLEANUP
// ====================================================
void cleanupSession() {

    if (motor_fd >= 0) {
        std::string stopMsg = "{\"action\":\"drive\",\"angle\":0,\"speed\":1500}\n";
        sendMotorCommand(motor_fd, stopMsg);
        close(motor_fd);
        motor_fd = -1;
    }

    if (lidar) {
        lidar->stop();          // OK
        lidar->disconnect();    // *** FIX CRITIQUE ***
        delete lidar;           // OK maintenant
        lidar = nullptr;
    }

    if (lidarChannel) {
        delete lidarChannel;
        lidarChannel = nullptr;
    }

    if (xbee_fd >= 0) {
        close(xbee_fd);
        xbee_fd = -1;
    }
}

// ====================================================
```

```
// MAIN
// ======================================================
int main() {
    signal(SIGINT, ctrlc_handler);

    if (gpioInitialise() < 0) {
        std::cerr << "pigpio init failed!" << std::endl;
        return 1;
    }

    const float FLOOR_OBSTACLE_THRESHOLD_CM = 30.0f;

    while (true) {

        if (stopRequested) break;

        std::cout << "\n===== NEW SESSION: waiting for START over XBee =====" <<
std::endl;

        const char* XBEE_DEV = "/dev/ttyUSB2";
        xbee_fd = open(XBEE_DEV, O_RDWR | O_NOCTTY | O_NONBLOCK);
        if (xbee_fd < 0) {
            std::cerr << "[XBEE] Impossible d'ouvrir " << XBEE_DEV
                      << " : " << strerror(errno) << std::endl;
            sleep(1);
            continue;
        }
        configSerialPort(xbee_fd, 9600);
        resetXbeeParser();

        std::string line;
        std::cout << "[XBEE] En attente du signal START..." << std::endl;

        bool gotStart = false;
        while (!gotStart) {
            if (stopRequested) break;

            if (readXbeeLine(xbee_fd, line)) {
                std::cout << "[XBEE] Ligne reçue : '" << line << "'" << std::endl;
                if (line == "START") {
                    std::cout << "[XBEE] START reçu → démarrage du robot !" << std::endl;
                    gotStart = true;
                }
            }
            usleep(10000);
        }

        if (stopRequested) break;
```

```
motor_fd = open("/dev/ttyUSB1", O_RDWR | O_NOCTTY | O_NONBLOCK);
if (motor_fd < 0) {
    std::cerr << "[MOTOR] Impossible d'ouvrir /dev/ttyUSB1 : "
              << strerror(errno) << std::endl;
    cleanupSession();
    sleep(1);
    continue;
}
configSerialPort(motor_fd, 115200);

// LIDAR INIT
auto channel_res = createSerialPortChannel("/dev/ttyUSB0", 256000);
if (!channel_res) {
    std::cerr << "[LIDAR] createSerialPortChannel failed" << std::endl;
    cleanupSession();
    sleep(1);
    continue;
}
lidarChannel = *channel_res;

auto driver_res = createLidarDriver();
if (!driver_res) {
    std::cerr << "[LIDAR] createLidarDriver failed" << std::endl;
    cleanupSession();
    sleep(1);
    continue;
}
lidar = *driver_res;

if (!lidar || !lidarChannel) {
    std::cerr << "[LIDAR] createLidarDriver or channel failed" << std::endl;
    cleanupSession();
    sleep(1);
    continue;
}

if (SL_IS_FAIL(lidar->connect(lidarChannel))) {
    std::cerr << "[LIDAR] connect failed" << std::endl;
    cleanupSession();
    sleep(1);
    continue;
}

lidar->reset(5000);
usleep(3000000);

lidar->setMotorSpeed();
```

```
lidar->startScan(false, false);
lidar->setMotorSpeed(200);

sl_lidar_response_measurement_node_hq_t nodes[8192];
size_t count = sizeof(nodes)/sizeof(nodes[0]);

int lastAngle = 0;
int lastSpeed = 1500;

bool sessionRunning = true;

while (sessionRunning) {
   if (stopRequested) break;

   unsigned long t0 = millis();

   unsigned long t_lidar_start = millis();

   float sum_dist[360] = {0};
   int   count_dist[360] = {0};

   count = sizeof(nodes)/sizeof(nodes[0]);

   sl_result res = lidar->grabScanDataHq(nodes, count, 50);

   float avg0 = 0;
   float maxValue = 0;
   int   maxAngle = 0;
   bool  hasData = false;

   if (res == SL_RESULT_OK && count > 0) {
      lidar->ascendScanData(nodes, count);

      for (size_t i = 0; i < count; ++i) {
         int angle = (nodes[i].angle_z_q14 * 90) >> 14;
         float dist = nodes[i].dist_mm_q2 / 4.0f;

         if ((angle >= 270 && angle < 360) || (angle >= 0 && angle <= 90)) {
            sum_dist[angle] += dist;
            count_dist[angle]++;
            hasData = true;
         }
      }

      for (int a = 0; a < 360; ++a) {
         if (count_dist[a] > 0) {
            float avg = sum_dist[a] / count_dist[a];
            if (a == 0) avg0 = avg;
```

```
                if (avg > maxValue) {
                    maxValue = avg;
                    maxAngle = a;
                }
            }
        }
    } else {
        std::cout << "[LIDAR] error: " << res << " (using last valid scan)" << std::endl;
    }

    unsigned long t_lidar_end = millis();

    int sendAngle = lastAngle;
    int speed = lastSpeed;

    if (hasData) {
        sendAngle = (avg0 > 500) ? 0 : maxAngle;

        if (avg0 > 500) speed = 1820;
        else if (avg0 >= 300) speed = 1780;
        else if (avg0 >= 200) speed = 1730;
        else if (avg0 >= 100) speed = 1700;
        else {
            speed = 1330;
            sendAngle = (sendAngle + 180) % 360;
        }

        lastAngle = sendAngle;
        lastSpeed = speed;
    }

    unsigned long t_ultra_start = millis();

    float floorDist = readGroveUltrasonic(22);
    bool floorObstacle = (floorDist > 0 && floorDist <
FLOOR_OBSTACLE_THRESHOLD_CM);
    bool maxanglesense= (maxAngle>=0 && 90>=maxAngle);
    if (floorObstacle) {
        if(sendAngle==0 && maxanglesense) sendAngle=90;
        else if(sendAngle==0&&!maxanglesense) sendAngle=270;
        std::cout << "[ULTRA] obstacle sol à " << floorDist << " cm | angle=" << sendAngle
<< std::endl;
        speed = 1700;
    } else {
        std::cout << "[LIDAR] dist_front=" << avg0
                  << " | angle=" << sendAngle
                  << " | speed=" << speed
                  << (speed < 1500 ? " (REVERSE)" : " (FORWARD)")
```

```
                << std::endl;
        }

        unsigned long t_ultra_end = millis();

        unsigned long t_rest_start = millis();

        {
            std::stringstream ss;
            ss << "{\"action\":\"drive\",\"angle\":" << sendAngle
               << ",\"speed\":" << speed << "}\n";

            std::string msg = ss.str();
            sendMotorCommand(motor_fd, msg);
        }

        if (readXbeeLine(xbee_fd, line)) {
            std::cout << "[XBEE] Ligne reçue : '" << line << "'" << std::endl;

            if (line == "END") {
                std::cout << "[XBEE] END reçu → arrêt immédiat !" << std::endl;
                sessionRunning = false;
            }
        }

        unsigned long t_rest_end = millis();
        unsigned long t1 = millis();

        std::cout << "[TIMING] lidar="  << (t_lidar_end  - t_lidar_start)
                  << " ms, ultra="      << (t_ultra_end  - t_ultra_start)
                  << " ms, rest="       << (t_rest_end   - t_rest_start)
                  << " ms, total="      << (t1 - t0)     << " ms"
                  << std::endl;

        unsigned long elapsed = t1 - t0;
        if (elapsed < (unsigned long)LOOP_PERIOD_MS) {
            usleep((LOOP_PERIOD_MS - elapsed) * 1000);
        }
    }

    std::cout << "[SESSION] END → cleanup and wait for next START" << std::endl;
    cleanupSession();

    usleep(200000);
}

// FIN : on termine pigpio UNE SEULE FOIS
```

```
    gpioTerminate();
    return 0;
}



#include <iostream>
#include <signal.h>
#include <unistd.h>
#include <fcntl.h>
#include <sstream>
#include <chrono>
#include <pigpio.h>
#include <string>
#include <cerrno>
#include <cstring>

#include <sys/ioctl.h>
#include <asm/termbits.h>

#include "sl_lidar.h"
#include "sl_lidar_driver.h"

using namespace sl;

// ====================================================
// GLOBALS
// ====================================================
ILidarDriver* lidar = nullptr;
sl::IChannel* lidarChannel = nullptr;
int motor_fd = -1;
int xbee_fd  = -1;

const int LOOP_PERIOD_MS = 50;

// Flag d'arrêt propre
volatile sig_atomic_t stopRequested = 0;

// ====================================================
// millis()
// ====================================================
unsigned long millis() {
    using namespace std::chrono;
    static steady_clock::time_point start = steady_clock::now();
    return (unsigned long)std::chrono::duration_cast<std::chrono::milliseconds>(
            std::chrono::steady_clock::now() - start)
        .count();
```

```
}

// ======================================================
// CONFIG SERIAL
// ======================================================
void configSerialPort(int fd, int baudrate) {
    struct termios2 tio;
    ioctl(fd, TCGETS2, &tio);

    tio.c_cflag &= ~CBAUD;
    tio.c_cflag |= BOTHER;
    tio.c_ispeed = baudrate;
    tio.c_ospeed = baudrate;

    tio.c_cflag &= ~CRTSCTS;
    tio.c_cflag |= CREAD | CLOCAL;
    tio.c_cflag &= ~PARENB;
    tio.c_cflag &= ~CSTOPB;
    tio.c_cflag &= ~CSIZE;
    tio.c_cflag |= CS8;

    tio.c_cc[VTIME] = 0;
    tio.c_cc[VMIN]  = 0;

    ioctl(fd, TCSETS2, &tio);
}

// ======================================================
// XBEE PARSER
// ======================================================
bool readXbeeLine(int fd, std::string& lineOut) {
    static std::string buffer;
    char c;

    while (read(fd, &c, 1) == 1) {

        if ((unsigned char)c < 0x09 || ((unsigned char)c > 0x0D && (unsigned char)c < 0x20)) {
            std::cout << "[XBEE RAW] octet non imprimable reçu → XBee peut être en mode
API" << std::endl;
        }

        if (c == '\r') continue;

        if (c == '\n') {
            if (!buffer.empty()) {
                lineOut = buffer;
                buffer.clear();
```

```
            return true;
        }
        continue;
    }

    buffer += c;

    if (buffer.size() > 256) {
        std::cout << "[XBEE] buffer overflow → reset" << std::endl;
        buffer.clear();
    }
  }

  return false;
}

void resetXbeeParser() {
  std::string dummy;
}

// =======================================================
// CTRL+C
// =======================================================
void ctrlc_handler(int) {
  stopRequested = 1;
}

// =======================================================
// ULTRASON
// =======================================================
float readGroveUltrasonic(int pin)
{
  gpioSetMode(pin, PI_OUTPUT);
  gpioWrite(pin, 0);
  gpioDelay(2);

  gpioWrite(pin, 1);
  gpioDelay(10);
  gpioWrite(pin, 0);

  gpioSetMode(pin, PI_INPUT);

  uint32_t start_wait = gpioTick();
  while (gpioRead(pin) == 0) {
    if (gpioTick() - start_wait > 30000) return -1;
  }

  uint32_t high_start = gpioTick();
```

```
    while (gpioRead(pin) == 1) {
        if (gpioTick() - high_start > 30000) return -1;
    }
    uint32_t high_end = gpioTick();

    uint32_t duration = high_end - high_start;
    return duration / 58.0f;
}

// ======================================================
// NON-BLOCKING MOTOR WRITE
// ======================================================
void sendMotorCommand(int fd, const std::string& msg) {
    if (fd < 0) return;
    ssize_t n = write(fd, msg.c_str(), msg.size());
    if (n < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            std::cout << "[MOTOR] write EAGAIN, skipping frame" << std::endl;
        } else {
            std::cout << "[MOTOR] write error: " << strerror(errno) << std::endl;
        }
    }
}

// ======================================================
// SESSION CLEANUP
// ======================================================
void cleanupSession() {

    if (motor_fd >= 0) {
        std::string stopMsg = "{\"action\":\"drive\",\"angle\":0,\"speed\":1500}\n";
        sendMotorCommand(motor_fd, stopMsg);
        close(motor_fd);
        motor_fd = -1;
    }

    if (lidar) {
        lidar->stop();          // OK
        lidar->disconnect();    // *** FIX CRITIQUE ***
        delete lidar;           // OK maintenant
        lidar = nullptr;
    }

    if (lidarChannel) {
        delete lidarChannel;
        lidarChannel = nullptr;
    }
```

```
    if (xbee_fd >= 0) {
        close(xbee_fd);
        xbee_fd = -1;
    }
}

// ====================================================
// MAIN
// ====================================================
int main() {
    signal(SIGINT, ctrlc_handler);

    if (gpioInitialise() < 0) {
        std::cerr << "pigpio init failed!" << std::endl;
        return 1;
    }

    const float FLOOR_OBSTACLE_THRESHOLD_CM = 30.0f;

    while (true) {

        if (stopRequested) break;

        std::cout << "\n===== NEW SESSION: waiting for START over XBee =====" <<
std::endl;

        const char* XBEE_DEV = "/dev/ttyUSB2";
        xbee_fd = open(XBEE_DEV, O_RDWR | O_NOCTTY | O_NONBLOCK);
        if (xbee_fd < 0) {
            std::cerr << "[XBEE] Impossible d'ouvrir " << XBEE_DEV
                      << " : " << strerror(errno) << std::endl;
            sleep(1);
            continue;
        }
        configSerialPort(xbee_fd, 9600);
        resetXbeeParser();

        std::string line;
        std::cout << "[XBEE] En attente du signal START..." << std::endl;

        bool gotStart = false;
        while (!gotStart) {
            if (stopRequested) break;

            if (readXbeeLine(xbee_fd, line)) {
                std::cout << "[XBEE] Ligne reçue : '" << line << "'" << std::endl;
                if (line == "START") {
                    std::cout << "[XBEE] START reçu → démarrage du robot !" << std::endl;
```

```
            gotStart = true;
        }
    }
    usleep(10000);
}

if (stopRequested) break;

motor_fd = open("/dev/ttyUSB1", O_RDWR | O_NOCTTY | O_NONBLOCK);
if (motor_fd < 0) {
    std::cerr << "[MOTOR] Impossible d'ouvrir /dev/ttyUSB1 : "
              << strerror(errno) << std::endl;
    cleanupSession();
    sleep(1);
    continue;
}
configSerialPort(motor_fd, 115200);

// LIDAR INIT
auto channel_res = createSerialPortChannel("/dev/ttyUSB0", 256000);
if (!channel_res) {
    std::cerr << "[LIDAR] createSerialPortChannel failed" << std::endl;
    cleanupSession();
    sleep(1);
    continue;
}
lidarChannel = *channel_res;

auto driver_res = createLidarDriver();
if (!driver_res) {
    std::cerr << "[LIDAR] createLidarDriver failed" << std::endl;
    cleanupSession();
    sleep(1);
    continue;
}
lidar = *driver_res;

if (!lidar || !lidarChannel) {
    std::cerr << "[LIDAR] createLidarDriver or channel failed" << std::endl;
    cleanupSession();
    sleep(1);
    continue;
}

if (SL_IS_FAIL(lidar->connect(lidarChannel))) {
    std::cerr << "[LIDAR] connect failed" << std::endl;
    cleanupSession();
    sleep(1);
```

```
    continue;
}

lidar->reset(5000);
usleep(3000000);

lidar->setMotorSpeed();
lidar->startScan(false, false);
lidar->setMotorSpeed(200);

sl_lidar_response_measurement_node_hq_t nodes[8192];
size_t count = sizeof(nodes)/sizeof(nodes[0]);

int lastAngle = 0;
int lastSpeed = 1500;

bool sessionRunning = true;

while (sessionRunning) {
    if (stopRequested) break;

    unsigned long t0 = millis();

    unsigned long t_lidar_start = millis();

    float sum_dist[360] = {0};
    int   count_dist[360] = {0};

    count = sizeof(nodes)/sizeof(nodes[0]);

    sl_result res = lidar->grabScanDataHq(nodes, count, 50);

    float avg0 = 0;
    float maxValue = 0;
    int   maxAngle = 0;
    bool  hasData = false;

    if (res == SL_RESULT_OK && count > 0) {
        lidar->ascendScanData(nodes, count);

        for (size_t i = 0; i < count; ++i) {
            int angle = (nodes[i].angle_z_q14 * 90) >> 14;
            float dist = nodes[i].dist_mm_q2 / 4.0f;

            if ((angle >= 270 && angle < 360) || (angle >= 0 && angle <= 90)) {
                sum_dist[angle] += dist;
                count_dist[angle]++;
                hasData = true;
```

```
            }
        }

        for (int a = 0; a < 360; ++a) {
            if (count_dist[a] > 0) {
                float avg = sum_dist[a] / count_dist[a];
                if (a == 0) avg0 = avg;
                if (avg > maxValue) {
                    maxValue = avg;
                    maxAngle = a;
                }
            }
        }
    } else {
        std::cout << "[LIDAR] error: " << res << " (using last valid scan)" << std::endl;
    }

    unsigned long t_lidar_end = millis();

    int sendAngle = lastAngle;
    int speed = lastSpeed;

    if (hasData) {
        sendAngle = (avg0 > 500) ? 0 : maxAngle;

        if (avg0 > 500) speed = 1680;
        else if (avg0 >= 300) speed = 1650;
        else if (avg0 >= 200) speed = 1630;
        else if (avg0 >= 100) speed = 1620;
        else {
            speed = 1350;
            sendAngle = (sendAngle + 180) % 360;
        }

        lastAngle = sendAngle;
        lastSpeed = speed;
    }

    unsigned long t_ultra_start = millis();

    float floorDist = readGroveUltrasonic(22);
    bool floorObstacle = (floorDist > 0 && floorDist <
FLOOR_OBSTACLE_THRESHOLD_CM);
    bool maxanglesense= (maxAngle>=0 && 90>=maxAngle);
    if (floorObstacle) {
        if(sendAngle==0 && maxanglesense) sendAngle=90;
        else if(sendAngle==0&&!maxanglesense) sendAngle=270;
```

```
            std::cout << "[ULTRA] obstacle sol à " << floorDist << " cm | angle=" << sendAngle
<< std::endl;
            speed = 1620;
        } else {
            std::cout << "[LIDAR] dist_front=" << avg0
                      << " | angle=" << sendAngle
                      << " | speed=" << speed
                      << (speed < 1500 ? " (REVERSE)" : " (FORWARD)")
                      << std::endl;
        }

        unsigned long t_ultra_end = millis();

        unsigned long t_rest_start = millis();

        {
            std::stringstream ss;
            ss << "{\"action\":\"drive\",\"angle\":" << sendAngle
               << ",\"speed\":" << speed << "}\n";

            std::string msg = ss.str();
            sendMotorCommand(motor_fd, msg);
        }

        if (readXbeeLine(xbee_fd, line)) {
            std::cout << "[XBEE] Ligne reçue : '" << line << "'" << std::endl;

            if (line == "END") {
                std::cout << "[XBEE] END reçu → arrêt immédiat !" << std::endl;
                sessionRunning = false;
            }
        }

        unsigned long t_rest_end = millis();
        unsigned long t1 = millis();

        std::cout << "[TIMING] lidar="  << (t_lidar_end  - t_lidar_start)
                  << " ms, ultra="      << (t_ultra_end  - t_ultra_start)
                  << " ms, rest="       << (t_rest_end   - t_rest_start)
                  << " ms, total="      << (t1 - t0)     << " ms"
                  << std::endl;

        unsigned long elapsed = t1 - t0;
        if (elapsed < (unsigned long)LOOP_PERIOD_MS) {
            usleep((LOOP_PERIOD_MS - elapsed) * 1000);
        }
    }
```

```
        std::cout << "[SESSION] END → cleanup and wait for next START" << std::endl;
        cleanupSession();

        usleep(200000);
    }

    // FIN : on termine pigpio UNE SEULE FOIS
    gpioTerminate();
    return 0;
}
```